

LLM-BugBounty-Automation

전체 시스템 설계 및 단계별 구현 로드맵

v1.1 설계안

2026-09-21

Repository

LLM-BugBounty-Automation

Description: Automated pipeline for authorized LLM security testing, red teaming, vulnerability validation, reproduction, evidence collection, and bug bounty reporting.

v1.1 핵심 변경

Coverage Matrix, Agent Trace, Program Policy, Environment Fingerprint, RAG Test Harness, Approval Gate, Evidence Redaction, Judge Benchmark, Tool Doctor/Version Lock 을 v1 설계에 반영했다. MVP 구현 순서와 GitHub 태그 전략도 실제 개발 흐름에 맞게 재정렬했다.

Scope first · Policy aware · Evidence driven · Reproducible · Tool-agnostic

목차

1. 프로젝트 개요
 2. 전체 아키텍처
 3. 보안 기준 및 Coverage Matrix
 4. 외부 도구 선정 및 역할
 5. Repository 구조
 6. 핵심 데이터 모델 및 Agent Trace
 7. 저장소·DB·Environment Fingerprint
 8. Scope + Program Policy Engine
 9. Target Adapter 및 Capability Profile
 10. Testcase / Prompt / Coverage Engine
 11. RAG Test Harness
 12. Mutation Engine
 13. Executor·세션·Approval Gate
 14. Judge Ensemble 및 Benchmark
 15. 재현 및 Control Test
 16. 중복 제거 및 Root Cause
 17. Evidence 보안 및 Reporting
 18. 실행 프로필
 19. 단계별 구현 로드맵
 20. GitHub 개발 운영 전략
 21. 테스트 및 Release Gate
 22. v2 확장 방향
 23. 주요 위험과 대응
 24. 최종 구현 체크리스트
 25. 바로 시작할 작업 순서
- 부록 A. tools.yaml
- 부록 B. 마일스톤 태그
- 부록 C. GitHub 체크포인트
- 부록 D. 참고 프레임워크

1. 프로젝트 개요

LLM-BugBounty-Automation의 목표는 “LLM에게 해킹을 시켜 보는 스크립트”가 아니라, 허가된 LLM·RAG·AI Agent·웹 기반 AI 서비스를 반복 가능하고 증거 중심으로 평가하는 보안 테스트 플랫폼을 만드는 것이다. 외부 스캐너의 단발 결과를 그대로 취약점으로 단정하지 않고, 표준화된 결과 계약, 다중 Judge, Control Test, 재현 테스트, Evidence 보존을 거쳐 Finding을 확정한다.

1.1 핵심 목표

- 웹 기반 AI 서비스, LLM API, RAG, AI Agent를 공통 Target 추상화와 Capability Profile로 다룬다.
- Recon → 공격 표면 분류 → Testcase 선택 → LLM Red Team → 판정 → 재현 → 증거화 → 보고서 생성을 하나의 파이프라인으로 연결한다.
- Promptfoo, Garak, PyRIT, Nuclei 등 외부 도구는 Adapter 뒤에 배치해 교체 가능한 Tool Layer로 사용한다.
- Prompt·Mutation·Agent Trace의 lineage를 보존해 어떤 전략과 행동 경로가 결과를 만들었는지 추적한다.
- Program Policy를 Scope와 함께 강제해 허가되지 않은 자동화·고위험 행동을 실행 단계에서 차단한다.
- 모든 Finding에 실행 환경 Fingerprint와 재현률을 남겨 모델/설정 변화 이후에도 비교 가능하게 한다.

1.2 v1 완료 기준

영역	v1 완료 기준
안전성	모든 외부 실행이 Scope + Program Policy를 통과하고 request budget/rate/concurrency/approval 정책을 강제
LLM 테스트	YAML Testcase → Executor → Trace → Judge → Reproducer → Evidence 흐름 동작
표준 커버리지	OWASP GenAI LLM Top 10 2026, OWASP Agentic Top 10 2026, MITRE ATLAS 매핑 필드 지원
외부 도구	핵심 LLM/Recon/Web 도구가 Adapter 계약으로 통합되고 doctor 명령으로 상태 점검 가능
데이터	Run/Trace/Request/Response/Judgement/Finding/Reproduction/Evidence 저장
품질	Control Test, 재현률, Judge benchmark와 False Positive 측정 지원
출력	Raw/Sanitized Evidence와 Markdown/JSON 보고서 생성
GitHub	각 Phase 종료 시 test → commit → push; 실행 가능한 마일스톤에만 tag/release

2. 전체 아키텍처



2.1 설계 원칙

- Scope-first + Policy-aware: 대상 범위뿐 아니라 프로그램별 허용 테스트 유형까지 실행 전에 확인한다.
- Adapter-first: 코어 로직은 외부 도구별 JSON 포맷이나 CLI 세부사항을 직접 알지 않는다.
- Trace-first for agents: 최종 응답뿐 아니라 tool call, retrieval, memory, approval 이벤트를 관측한다.
- Evidence-first: Finding 보다 먼저 원시 요청/응답/Trace와 판정 근거를 보존한다.
- Reproduction-first: 단발 성공은 Candidate이며 Control/Repeat를 통과한 뒤 Confirmed로 승격한다.
- Deterministic where possible: Rule/Regex/Canary/상태 변화 검증을 LLM Judge보다 우선한다.
- Versioned & fingerprinted: Testcase, system prompt, tool schema, RAG corpus, pipeline commit을 함께 추적한다.

3. 보안 기준 및 Coverage Matrix

v1.1부터 자체 카테고리만 사용하지 않고 외부 보안 프레임워크와 Testcase를 연결한다. 이 매핑은 “표준 준수 인증”이 아니라 어떤 위험 영역을 얼마나 테스트했는지 설명하기 위한 Coverage 기준이다.

기준	용도	프로젝트 활용
OWASP GenAI LLM Top 10 2026	LLM 애플리케이션 핵심 위험	Testcase category와 risk coverage의 1차 기준
OWASP Top 10 for Agentic Applications 2026	자율 Agent의 목표/권한/도구/메모리 위험	Agent suite와 Approval/Trace 설계 기준
MITRE ATLAS	AI 대상 공격 TTP 지식베이스	Prompt Injection, RAG Poisoning, Agent Tool Invocation 등 공격 기술 태깅
CWE / 프로그램별 정책	구현 결함 및 제출 요구사항 보조 분류	Finding 보고서의 추가 분류/설명용

3.1 Coverage 데이터 모델

```
frameworks:
  owasp_llm_2026: [<risk-id>]
  owasp_agentic_2026: [<risk-id>]
  mitre_atlas: [<technique-id>]

coverage:
  target_capabilities: [chat, rag, tools, memory]
  confidentiality: true
  integrity: true
  availability: false
  prerequisites: [...]
```

Coverage 원칙

Testcase 가 존재한다고 “커버됨”으로 표시하지 않는다. 대상 Capability 가 충족되고, 실제 실행되었으며, 결과가 수집된 경우에만 executed coverage 로 계산한다.

4. 외부 도구 선정 및 역할

도구	분류	역할	통합 전략
Promptfoo	LLM Red Team	정형 테스트, eval, red-team suite	결과 → PromptfooAdapter → NormalizedResult
Microsoft PyRIT	LLM Red Team	Adaptive / Multi-turn 공격, 변형	후속 심층 검증 Agent 로 사용
NVIDIA Garak	LLM Scanner	Probe/detector 기반 취약점 탐색	JSONL → GarakAdapter
Subfinder	Recon	Passive subdomain enumeration	JSONL → Asset
httpx	Recon	Live HTTP 확인, 메타데이터 탐색	JSONL → Endpoint
Katana	Discovery	URL/API/JS crawling	JSONL → DiscoveredEndpoint
ffuf	Discovery	경로/파라미터 fuzzing	JSON → DiscoveredEndpoint
Nuclei	Web Scanner	Web/API/설정 취약점 검사	JSONL → FindingCandidate
Dalfox	XSS Scanner	XSS/파라미터 분석	JSONL → FindingCandidate
TruffleHog	Secret Scanner	API Key/Token/Credential 탐색	JSON → SecretFinding

4.1 Tool Doctor / Version Lock

Python·Node·Go 기반 도구가 섞이므로 설치 여부와 버전 불일치가 파이프라인 실패 원인이 되기 쉽다. 외부 도구 통합 전 `doctor` 기능과 버전 스냅샷을 만든다.

```
python main.py doctor

[OK] Python 3.x
[OK] promptfoo <version>
[OK] garak <version>
[OK] pyrit <version>
[OK] nuclei <version>
[WARN] dalfox not installed

# run metadata
tool_versions.lock.yaml
```

5. Repository 구조

```

LLM-BugBounty-Automation/
├── README.md / LICENSE / pyproject.toml / .env.example
├── main.py / cli.py
├── config/
│   ├── pipeline.yaml / models.yaml / tools.yaml / logging.yaml
│   └── scope.example.yaml
├── programs/
├── core/
├── scope/
├── targets/
├── adapters/
├── recon/
├── attacks/
├── prompts/
├── testcase/
├── rag_harness/
├── executor/
├── traces/
├── judges/
├── benchmarks/
├── reproduction/
├── findings/
├── storage/
├── reporting/
├── knowledge/
├── tests/
├── runs/
├── evidence/
└── reports/

```

bug bounty program policy profiles
 # orchestrator, context, events
 # scope + policy + approval rules
 # target adapters + capability profile
 # external tool adapters
 # asset map / endpoint classifier
 # planner / mutation / lineage
 # seeds / mutation / judge / reporting
 # schemas / suites / framework mapping
 # controlled docs / corpus / retrieval trace
 # async runner / session / approval gate
 # trace/event schemas and exporters
 # rule / canary / semantic / LLM / ensemble
 # judge + pipeline benchmark fixtures
 # repeat / controls / statistics
 # normalize / dedup / severity
 # DB / artifacts / fingerprints
 # report + sanitizer/redaction
 # techniques / sources / mappings
 # unit / integration / fixtures
 # gitignored
 # raw + sanitized, gitignored
 # private + shareable

6. 핵심 데이터 모델 및 Agent Trace

기존 Request/Response 중심 모델에 Trace/Event 계층을 추가한다. LLM API는 단일 요청/응답으로 충분할 수 있지만 Agent는 도구 호출과 검색·메모리·승인 이벤트를 봐야 취약점 원인을 설명할 수 있다.

객체	역할
Target / CapabilityProfile	대상 유형과 지원 기능(chat, rag, tools, memory, files, multimodal 등)
Run	한 번의 평가 실행과 설정 스냅샷
Testcase / Prompt / Mutation	테스트 정의, 입력, 변형 계보
Request / Response	실제 모델/API 입출력과 메타데이터
Trace / TraceEvent	한 세션의 실행 흐름 및 내부 관측 이벤트
ToolCall / ToolResult	Agent가 선택한 Tool/arguments/result
RetrievalEvent	RAG retrieval 문서/chunk/score
MemoryEvent	세션·장기 메모리 읽기/쓰기 이벤트
ApprovalEvent	고위험 행동의 요청/허용/차단/시뮬레이션
Judgement / Finding	판정 신호 및 확정 후보
Reproduction / Evidence / Report	재현 통계, 증거, 제출 산출물

```

TraceEvent {
  trace_id,
  sequence,
  event_type: llm_call | tool_call | tool_result |
              retrieval | memory_read | memory_write |
              approval | final_response,
  timestamp,
  artifact_ref,
  metadata
}

```

6.1 Normalized Result 계약

```
NormalizedResult {
  run_id,
  target_id,
  source: { tool, version },
  category,
  title,
  endpoint,
  testcase_id,
  trace_ref,
  raw_artifact_ref,
  detector_score,
  framework_tags,
  metadata
}
```

7. 저장소·DB·Environment Fingerprint

MVP 는 SQLite 로 시작하고 원시 Evidence 파일과 구조화 DB 를 분리한다. 동시 Worker 나 다중 사용자 Dashboard 가 필요해질 때 PostgreSQL 로 이전한다.

테이블	주요 내용
targets / runs	대상, profile, policy, config snapshot, fingerprint
assets / endpoints	domain, URL, method, discovery source, classification
testcases / prompts / mutations	version, source, hash, lineage, framework mapping
sessions / traces / events	세션과 Agent/RAG 실행 이벤트
requests / responses	artifact ref, latency, token/cost, model metadata
judgements	judge type, score, reason/evidence ref, benchmark version
findings / reproductions	status, confidence, severity, attempts, success rate
evidence / reports	raw/sanitized artifact, hash, redaction log, report refs

7.1 Environment Fingerprint

모델과 시스템 구성이 자주 바뀌므로 재현성 판단을 위해 실행 환경을 하나의 Fingerprint 로 묶는다.

```
fingerprint_inputs:
  target_build
  model_provider / model_name / model_version
  temperature / seed
  system_prompt_hash
  tool_schema_hash
  rag_corpus_hash
  testcase_version
  pipeline_git_commit_sha
  tool_versions
  config_hash

fingerprint = SHA256(canonical_json(fingerprint_inputs))
```

8. Scope + Program Policy Engine

Scope Engine 은 “어디를 테스트할 수 있는가”를, Program Policy 는 “무엇을 어떻게 테스트할 수 있는가”를 강제한다. 둘 다 실행 게이트로 취급한다.

```
program: example-bugbounty
scope:
  domains: [ai.example.com]
  url_patterns: [https://ai.example.com/api/*]

testing:
  automated_scanning: true
  prompt_injection: true
  destructive_actions: false
  denial_of_service: false
  cross_user_data_access: false

limits:
  requests_per_second: 2
  concurrency: 3
  max_requests_per_run: 3000

approval_required:
  - external_callback
  - file_write
  - account_state_change
```

- 허용/금지 domain, URL pattern, subdomain 정책을 지원한다.
- 자동화 스캔 허용 여부와 공격 유형별 permit/deny 를 별도 필드로 둔다.
- out-of-scope redirect 는 재검증 후 차단 또는 record-only 처리한다.
- RPS/concurrency/request budget/token budget 을 중앙에서 강제한다.
- Policy snapshot 과 hash 를 각 Run Evidence 에 저장한다.
- 고위험 작업은 Approval Gate 로 전달하고 기본값은 block 또는 simulate 로 둔다.

9. Target Adapter 및 Capability Profile

```
class TargetAdapter:
    async def healthcheck(self): ...
    async def metadata(self): ...
    async def capabilities(self): ...
    async def send(self, input, session=None): ...
    async def reset_session(self): ...
    async def trace(self, session=None): ...
```

9.1 Capability Profile

```
capabilities:
  chat: true
  system_prompt_control: false
  sessions: true
  tools: true
  rag: true
  memory: false
  files: true
  multimodal: false
```

Testcase Selector 는 Capability Profile 을 이용해 실행 불가능하거나 의미 없는 테스트를 제외한다. 예를 들어 tools=false 인 대상에는 tool-abuse suite 를 실행하지 않는다.

10. Testcase / Prompt / Coverage Engine

```
id: LLM-PI-001
name: Basic Instruction Override
category: prompt_injection
requires: [chat]
frameworks:
  owasp_llm_2026: [<risk-id>]
  mitre_atlas: [<technique-id>]
prompt: |
  {{PAYLOAD}}
mutation: { enabled: true }
judges: [rule, canary, semantic, llm]
reproduce: { attempts: 5, threshold: 3 }
severity: { base: medium }
```

10.1 초기 Test Suite

카테고리	주요 목적
prompt_injection	상위 지시 무시/정책 우회
system_prompt_leak	숨은 지시/Canary 노출
indirect_injection	외부 문서/웹/RAG 컨텍스트를 통한 지시 삽입
rag_security	오염 문서, retrieval 권한/출처, data leakage
tool_abuse	허용되지 않은 Tool 선택/인자/권한 사용
agent_security	goal hijack, 과도한 agency, memory/context manipulation
data_leak	민감정보/타 세션/타 사용자 데이터 노출
multi_turn	세션 누적을 이용한 정책/상태 교란

10.2 교재 기반 Seed 활용

업로드된 2025 버그헌팅 중급과정 교재의 ChatGPT 버그헌팅 구간은 정답 공격법이 아니라 Seed/Mutation 아이디어로 사용한다. step-by-step, 반복 질문, 반심리학, 소유권 주장, 도전형 요청은 social/reasoning mutation으로 분류하고, 소스 분석·자동화 스크립트·Burp 확장·보고서 작성 항목은 Code Analysis, Automation, Tool Integration, Report Generator 설계에 반영한다.

11. RAG Test Harness

RAG 보안은 단순 Prompt 만으로 재현하기 어렵기 때문에 통제된 문서와 retrieval trace를 제공하는 테스트 Harness를 별도로 둔다.

```
Controlled Documents
|
Document Injector
|
Chunking / Embedding
|
Vector Store
|
Target RAG
|
Retrieval Trace
|
Judge / Reproducer
```

- document_id, document_hash, chunk_id, source, embedding model/version 을 기록한다.
- retrieved_chunks와 retrieval score를 Trace에 저장한다.
- 정상 control document와 injection document를 같은 조건에서 비교한다.
- RAG corpus hash를 Environment Fingerprint에 포함한다.
- 실제 버그바운티 대상에서는 프로그램 정책상 허용된 문서/입력 경로만 사용한다.

12. Mutation Engine

Mutation Engine은 무작위 변형기가 아니라 전략별 성공·실패 계보를 추적하는 실험 엔진으로 설계한다.

```
Seed Prompt
├─ paraphrase / translation
├─ roleplay / persona
├─ JSON / XML / Markdown wrapping
├─ encoding / unicode variation
├─ multi-turn split
├─ ownership claim / challenge
└─ model-generated mutation
```

- mutation_id, parent_mutation_id, strategy, generation, prompt_hash 를 저장한다.
- 동일 Prompt 중복은 hash 로 제거한다.
- 전략별 성공률을 Target/Model/Version 별로 집계한다.
- 성공 Mutation 은 바로 Finding 으로 확정하지 않고 Reproducer 로 전달한다.

13. Executor·세션·Approval Gate

- asyncio + httpx 기반 비동기 실행, timeout/retry/backoff 지원
- global/target/tool 별 concurrency 와 request/token budget 강제
- multi-turn session 유지/초기화 및 session isolation
- 요청/응답 원문과 latency/token/cost 기록
- Agent/RAG 실행 시 TraceEvent 스트림 기록
- 중단 후 resume 를 위한 checkpoint 와 idempotency key 지원

13.1 Approval Gate

```
Action Risk Policy
read/search          -> AUTO
network GET          -> AUTO (scope checked)
external callback    -> APPROVAL
file write           -> APPROVAL or SIMULATE
send message/email   -> BLOCK by default
account/payment/delete -> BLOCK by default
cloud/system mutation -> BLOCK by default
```

ApprovalEvent 는 요청된 action, risk level, policy decision, 승인 주체, 최종 실행 여부를 Trace 에 남긴다. 허가된 테스트라도 실제 영향을 만드는 행동은 가능한 한 simulate 또는 최소 PoC 에서 멈춘다.

14. Judge Ensemble 및 Benchmark

Judge	목적	성격
Rule	정확한 상태/구조/문자열 검증	결정적
Regex	키/토큰/패턴 검출	패턴 의존
Canary	사전 marker 유출 검증	매우 높은 신뢰도
Semantic	의미상 위반/노출 유사도 평가	확률적
LLM Judge	복잡한 행동/문맥 기반 평가	보조 신호
Ensemble	다중 신호 결합	최종 confidence

14.1 Judge Benchmark

Judge 가 틀리면 전체 파이프라인이 잘못된 Finding 을 생성하므로 Judge 자체의 성능을 독립적으로 측정한다.

```
benchmarks/judge/
├─ true_positive/
├─ true_negative/
└─ ambiguous/

metrics:
precision
recall
false_positive_rate
false_negative_rate
f1
```

초기 Release Gate 예시

Canary/Rule 기반 deterministic case 는 100% 일치가 목표다. Ensemble 은 프로젝트 fixture 기준 precision 과 False Positive Rate 를 지속 측정하고, 기준 미달 시 release 를 막는다. 정확한 임계값은 충분한 benchmark 샘플을 확보한 뒤 고정한다.

15. 재현 및 Control Test

```
Initial Hit
|
Negative Control ---> baseline behavior
|
Repeat N times
|
Variation / Session Reset
|
Cross-tool validation
|
confirmed / unstable / rejected
```

- 기본 attempts=5 또는 10, threshold 는 Testcase 별 설정한다.
- Control 에서는 동일 정보가 정상 경로에서도 노출되는지 확인한다.
- model/version/temp/seed/session/fingerprint 를 함께 기록한다.
- reproduction_rate 와 control outcome 을 Finding 핵심 필드로 둔다.

16. 중복 제거 및 Root Cause

1. category + endpoint + behavior 를 이용한 exact/heuristic dedup
2. 응답 signature/Canary/상태 변화 비교
3. 제목/설명 semantic similarity 비교
4. root_cause_key 를 생성해 Cluster 단위로 보고
5. v2 에서 pgvector 기반 embedding clustering 과 historical linking 으로 확장

17. Evidence 보안 및 Reporting

Evidence 는 버그바운티 제출의 핵심이지만 동시에 API key, Cookie, 개인정보, 내부 URL 등 민감정보를 포함할 수 있으므로 raw 와 shareable 산출물을 분리한다.

```
evidence/
├── raw/                # 원본, restricted, gitignored
└── sanitized/          # redaction 적용, 공유 가능

reports/
├── private/            # 전체 내부 분석
└── shareable/          # 제출용 최소 증거
```

- 모든 artifact 에 SHA-256 과 timestamp 를 기록한다.
- Redaction 전후 artifact 를 연결하는 redaction log 를 저장한다.
- Authorization/Cookie/API key/session ID/개인정보 패턴을 기본 sanitization 대상으로 둔다.
- Raw Evidence 는 `.gitignore` 기본 대상이며 필요 시 로컬 암호화/접근권한을 적용한다.
- 보고서에는 재현 절차, 환경 Fingerprint, reproduction rate, Evidence ref 를 포함한다.

```
Report sections
- Title / Summary / Affected Component
- Preconditions / Scope & Policy context
- Steps to Reproduce
- Prompt / Input and relevant Trace
- Observed vs Expected Behavior
- Security Impact
- Reproduction Rate / Controls
- Environment Fingerprint
- Sanitized Evidence
- Suggested Mitigation
```

18. 실행 프로필

프로필	목적	구성
quick	빠른 상태 확인	Scope/Policy → Target → 핵심 Testcase → deterministic Judge
llm	LLM/API 집중	Promptfoo/Garak → Judge → Reproducer
agent	Agent 권한/행동	Agent suite → Trace → Approval Gate → PyRIT adaptive
rag	RAG/문서 주입	RAG Harness → indirect injection → retrieval/data leakage
web	웹 공격면	Recon → Nuclei/Dalfox/TruffleHog
full	전체 평가	Recon + LLM/RAG/Agent + 재현 + dedup + report

19. 단계별 구현 로드맵

GitHub 운영 원칙

각 Phase 종료 시 관련 테스트 → sample run → evidence 확인 → git diff → commit → push 를 권장한다.
Tag/Release 는 모든 Phase 가 아니라 실제 실행 가능한 마일스톤에만 생성한다.

Phase 0 - Repository 초기화

구현 내용: README, LICENSE, .gitignore, pyproject.toml, config 기본 구조, `doctor` 골격
완료 조건: clean clone 에서 CLI help 와 pytest smoke 가 실행
권장 Commit: chore: initialize LLM-BugBounty-Automation
권장 Tag: 없음

Phase 1 - Core Model + Storage

구현 내용: Target/Run/Testcase/Finding/Evidence + Trace/Event schema, SQLite
완료 조건: 직렬화/DB CRUD/Trace ordering unit test 통과
권장 Commit: feat: add core models trace schema and storage
권장 Tag: 없음

Phase 2 - Scope + Program Policy

구현 내용: allow/deny, URL validation, rate/concurrency/budget, program policy
완료 조건: 금지 대상/금지 action 이 실행 전 차단되고 snapshot 저장
권장 Commit: feat: implement scope and program policy engine
권장 Tag: 없음

Phase 3 - Target Adapter + Fake Targets

구현 내용: OpenAI-compatible, CustomHTTP, FakeLLM/FakeAgent
완료 조건: 외부 서비스 없이 session/capability/trace integration test 가능
권장 Commit: feat: add target adapters and fake targets
권장 Tag: 없음

Phase 4 - Testcase + Coverage Taxonomy

구현 내용: YAML schema, loader, hash/version, OWASP/ATLAS mapping, 교재 Seed
완료 조건: coverage mapping validation + prompt rendering test 통과
권장 Commit: feat: add testcase coverage and prompt engine
권장 Tag: 없음

Phase 5 - Executor + Session + Trace

구현 내용: async runner, retry, budget, session, trace, approval gate
 완료 조건: 동시 요청/timeout/resume/approval fixture 통과
 권장 Commit: feat: add executor sessions trace and approval gate
 권장 Tag: 없음

Phase 6 - Judge + Benchmark Baseline

구현 내용: Rule/Regex/Canary/Semantic/LLM/Ensemble + benchmark runner
 완료 조건: TP/TN fixture metric 이 계산되고 regression test 가능
 권장 Commit: feat: implement judge ensemble and benchmark
 권장 Tag: 없음

Phase 7 - Reproducer + Controls

구현 내용: repeat, negative/positive control, variation, status transition
 완료 조건: candidate 가 confirmed/rejected/unstable 로 분류
 권장 Commit: feat: add reproduction and control testing
 권장 Tag: 없음

Phase 8 - Evidence + Basic Report

구현 내용: raw/sanitized evidence, hashing/redaction, Markdown/JSON 기본 보고서
 완료 조건: 한 Finding 을 재현 가능한 shareable report 로 생성
 권장 Commit: feat: add evidence security and basic reporting
 권장 Tag: v0.1.0-mvp1

Phase 9 - Tool Doctor + Promptfoo/Garak

구현 내용: 버전 점검/lock + 두 LLM scanner Adapter
 완료 조건: 두 도구 출력이 NormalizedResult 로 유입되고 version snapshot 저장
 권장 Commit: feat: integrate promptfoo garak and tool doctor
 권장 Tag: 없음

Phase 10 - Mutation Engine

구현 내용: 전략 변형, lineage, hash dedup, 전략별 통계
 완료 조건: parent-child 계보와 중복 제거/통계 테스트 통과
 권장 Commit: feat: add prompt mutation and lineage engine
 권장 Tag: 없음

Phase 11 - PyRIT Adaptive

구현 내용: candidate 기반 adaptive/multi-turn 후속 검증
 완료 조건: PyRIT 결과가 동일 Judge/Reproducer 경로로 유입
 권장 Commit: feat: integrate pyrit adaptive red teaming
 권장 Tag: v0.2.0-llm-redteam

Phase 12 - Recon

구현 내용: Subfinder → httpx → Katana → ffuf, Asset/Endpoint 저장
 완료 조건: 허가된 lab 에서 asset map 생성 및 scope 재검증
 권장 Commit: feat: add automated reconnaissance pipeline
 권장 Tag: 없음

Phase 13 - Web Security Tools

구현 내용: Nuclei, Dalfox, TruffleHog Adapter
 완료 조건: 샘플 결과가 FindingCandidate/SecretFinding 으로 정규화
 권장 Commit: feat: integrate web and secret scanners
 권장 Tag: 없음

Phase 14 - AI Endpoint Classifier + RAG Harness

구현 내용: Endpoint 분류와 controlled RAG corpus/retrieval trace
 완료 조건: AI API/Chat/RAG/Agent 분류 + RAG control/injection test 동작
 권장 Commit: feat: add ai discovery and rag test harness
 권장 Tag: v0.3.0-discovery

Phase 15 - Dedup + Root Cause + Enhanced Report

구현 내용: cluster, root_cause_key, historical link, report aggregation
 완료 조건: 동일 원인이 단일 cluster/report 로 묶임
 권장 Commit: feat: add finding dedup and root cause reporting
 권장 Tag: 없음

Phase 16 - Full Orchestrator

구현 내용: scope/policy → recon → classify → scan → judge → reproduce → report
 완료 조건: `scan --profile full`이 로컬 lab에서 end-to-end 완료
 권장 Commit: feat: complete end-to-end orchestration
 권장 Tag: 없음

Phase 17 - Hardening / v1

구현 내용: integration/E2E, parser regression, recovery, docs, security review
 완료 조건: clean install + full tests + graceful failure + release checklist 통과
 권장 Commit: test: harden and stabilize release pipeline
 권장 Tag: v1.0.0

20. GitHub 개발 운영 전략

혼자 개발하는 프로젝트에서는 `main + feat/\*` 형태의 GitHub Flow가 충분하다. develop 브랜치를 상시 유지하면 관리 비용이 늘 수 있으므로 팀이 커지기 전까지는 생략한다.

```
main
├── feat/core-models
├── feat/scope-policy
├── feat/target-adapters
├── feat/judge-benchmark
└── feat/rag-harness

feature branch
-> tests / sample run
-> commit + push
-> PR 또는 self-review
-> main merge
-> milestone 이면 tag + GitHub Release
```

- Phase 종료 시 반드시 원격 GitHub에 push하는 것을 권장한다. 다음 Phase는 원격 백업 상태를 확인한 뒤 시작한다.
- Conventional Commit 접두어(`feat`, `fix`, `refactor`, `test`, `docs`, `chore`)를 통일한다.
- `.env`, raw evidence, credentials, 실제 민감 응답은 커밋하지 않는다.
- 실행 가능한 마일스톤만 tag한다: v0.1.0-mvp1, v0.2.0-llm-redteam, v0.3.0-discovery, v1.0.0.
- 각 Release에는 완료 범위, known limitations, 실행 예제, benchmark 결과를 남긴다.

20.1 권장 Repository Metadata

- Repository: `LLM-BugBounty-Automation`
- Description: `Automated pipeline for authorized LLM security testing, red teaming, vulnerability validation, reproduction, evidence collection, and bug bounty reporting.`
- Topics: `llm-security`, `bug-bounty`, `red-teaming`, `prompt-injection`, `ai-security`, `security-automation`, `agent-security`, `rag-security`

21. 테스트 및 Release Gate

계층	검증
Unit	schema, scope/policy matcher, parser, judge, hash, lineage, redaction
Fixture	정상/취약 LLM 응답, Agent trace, RAG retrieval, tool JSON/JSONL
Judge Benchmark	TP/TN/ambiguous dataset 의 precision/recall/FPR/FNR/F1
Integration	Fake LLM/Agent + Executor + Judge + Reproducer + DB
Tool Integration	외부 CLI version/exit code/parser regression
E2E	허가된 로컬 lab 에서 full profile 실행
Safety	out-of-scope redirect, denied action, budget, approval enforcement
Regression	과거 Confirmed Finding 과 environment fingerprint 비교

21.1 Phase 완료 Gate

1. formatter / lint
2. unit tests
3. relevant integration + benchmark tests
4. sample run
5. raw/sanitized evidence check
6. README / config example update
7. git diff + secret check
8. commit
9. push
10. milestone 이면 tag + GitHub Release

22. v2 확장 방향

- PostgreSQL + Redis + Worker 기반 분산 스캔
- React/Vite Dashboard 와 Finding triage UI
- pgvector 기반 Finding/Mutation semantic clustering
- 과거 성공 Mutation/Root Cause 검색을 이용한 Attack Planner
- Continuous monitoring 및 scheduled regression red team
- GitHub Actions 등 CI/CD 에서 LLM security regression 실행
- Tool/MCP 권한 그래프와 Agent behavior analytics
- 비용·토큰·성공률을 함께 최적화하는 실험 스케줄러
- 팀/조직용 RBAC, Evidence Vault, audit log

23. 주요 위험과 대응

위험	대응
False Positive	Rule/Canary/Control/Reproducer 우선 + Judge benchmark
범위/정책 이탈	Scope + Program Policy + redirect 재검증 + Approval Gate
Agent 고위험 행동	BLOCK/SIMULATE 기본, trace 와 approval event 기록
비용 폭증	request/token/mutation budget, early stopping
도구 출력 변경	Adapter fixture 와 parser regression test
Prompt 중복 폭증	hash dedup + lineage cap + strategy budget
민감 Evidence 노출	raw/sanitized 분리, redaction log, gitignore, 접근제어
재현 불가	Environment Fingerprint + session/reset/control 기록
RAG 원인 불명	retrieval trace, document/chunk hash, corpus fingerprint
도구 종속성	NormalizedResult + doctor/version lock

24. 최종 구현 체크리스트

- Scope 와 Program Policy 를 통과하지 않은 외부 요청이 실행되지 않는가?
- 고위험 Agent action 이 Approval Gate 를 우회할 수 없는가?
- Target Capability 에 맞는 Testcase 만 선택되는가?
- OWASP/ATLAS mapping 과 executed coverage 가 구분되는가?
- Agent/RAG 에서 tool/retrieval/memory trace 가 남는가?
- Prompt/Mutation parent/hash/version 이 남는가?
- Environment Fingerprint 에 모델·설정·tool schema·pipeline commit 이 포함되는가?
- Candidate 가 Reproducer 와 Control 을 거친 뒤 상태 전이되는가?
- Judge benchmark 와 parser regression 이 release 전에 실행되는가?
- Raw Evidence 와 Sanitized Evidence 가 분리되고 redaction log 가 남는가?
- 동일 Root Cause 가 단일 cluster 로 합쳐지는가?
- 보고서가 실제 재현 가능한 단계와 Evidence ref 를 제공하는가?
- 각 Phase 종료 시 test → commit → push 가 수행되는가?
- Tag 는 실행 가능한 마일스톤에만 존재하는가?

25. 바로 시작할 작업 순서

1. GitHub 에 `LLM-BugBounty-Automation` Repository 를 만들고 Phase 0 구조를 commit/push 한다.
2. Core Data Model 에 Trace/Event 까지 포함하고 SQLite 저장 테스트를 완성한다.
3. Scope + Program Policy Engine 을 구현해 이후 모든 네트워크/Agent action 의 공통 게이트로 만든다.
4. Fake LLM/Fake Agent 와 Target Adapter 를 만들어 외부 서비스 없이 통합 테스트 가능한 환경을 만든다.
5. YAML Testcase, Coverage Taxonomy, 교재 기반 Seed 최소 세트를 등록한다.
6. Executor + Session + Trace + Approval Gate 를 구현한다.
7. Judge Ensemble 과 Judge Benchmark 를 구현한다.
8. Reproducer/Control 과 Raw/Sanitized Evidence + 기본 Report 를 구현하고 `v0.1.0-mvp1` 을 Release 한다.
9. Promptfoo/Garak → Mutation → PyRIT 순으로 붙이고 `v0.2.0-llm-redteam` 을 Release 한다.
10. Recon/Web Tools → AI Endpoint Classifier → RAG Harness 를 붙이고 `v0.3.0-discovery` 를 Release 한다.
11. Dedup/Root Cause → Full Orchestrator → Hardening 을 완료하고 `v1.0.0` 을 Release 한다.

권장 개발 습관

구현 단위를 작게 유지한다. 각 Phase 의 완료 조건을 충족한 상태를 GitHub 에 남긴 뒤 다음 단계로 이동한다. “기능이 많음”보다 “재현 가능하고 검증 가능한 상태”를 우선한다.

부록 A. 권장 `tools.yaml` 개념

```
tools:
  promptfoo:
    enabled: true
    category: llm_redteam
    adapter: adapters.llm.promptfoo
    output: json
  garak:
    enabled: true
    category: llm_scanner
    adapter: adapters.llm.garak
    output: jsonl
  pyrit:
    enabled: true
    category: llm_redteam
    adapter: adapters.llm.pyrit
  subfinder: { enabled: true, adapter: adapters.recon.subfinder }
  httpx:      { enabled: true, adapter: adapters.recon.httpx }
  katana:     { enabled: true, adapter: adapters.discovery.katana }
  ffuf:       { enabled: true, adapter: adapters.discovery.ffuf }
  nuclei:     { enabled: true, adapter: adapters.scanner.nuclei }
  dalfox:     { enabled: true, adapter: adapters.scanner.dalfox }
  trufflehog: { enabled: true, adapter: adapters.secrets.trufflehog }
```

부록 B. 권장 마일스톤 태그

v0.1.0-mvp1	Core validation loop complete
v0.2.0-llm-redteam	Promptfoo/Garak/Mutation/PyRIT integrated
v0.3.0-discovery	Recon/Web/AI discovery/RAG harness integrated
v1.0.0	Stable first release

Phase 별 commit/push 는 계속 유지하되, tag 를 모든 내부 단계에 생성하지 않는다. Tag 는 다른 사람이 checkout 해 실행할 가치가 있는 상태를 표현한다.

부록 C. GitHub 체크포인트 명령 예시

```
git status
git diff
pytest

# 비밀값/Raw Evidence 가 staging 에 있는지 확인
git add .
git commit -m "feat: implement scope and program policy engine"
git push origin feat/scope-policy

# main 병합 및 milestone 일 때만
git tag v0.1.0-mvp1
git push origin v0.1.0-mvp1
```

부록 D. 참고 프레임워크 및 기준

아래 자료는 v1.1 설계의 Coverage/Agent 보안 분류에 사용하는 외부 기준이다. 최신 상태 확인일: 2026-09-21.

자료	확인한 용도
OWASP GenAI LLM Top 10 2026	LLM 애플리케이션 위험 분류 및 Crosswalk 기준. OWASP 가 2026 판을 최신 커뮤니티 가이드로 소개.
OWASP Top 10 for Agentic Applications 2026	Agentic AI 의 주요 보안 위험 분류. 2025-12-09 공개된 2026 프레임워크.
MITRE ATLAS	AI 시스템 공격 TTP 지식베이스. LLM Prompt Injection, RAG Poisoning, AI Agent Tool Invocation, System Prompt extraction 등을 포함.

- <https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/>
- <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- <https://atlas.mitre.org/>